Device API & JSON Reference

Smart Medication Dispenser — Firmware / Electronics

ENGINEERING HANDOFF

Document: DEVICE_API_FIRMWARE_GUIDE.md

Content: Per-endpoint *what it does*, *why*, *when to call*; JSON examples; field tables

Regenerate: `python generate_device_api_pdf.py` (updates this HTML + PDF)

Device API & JSON Reference — Firmware / Electronics

Smart Medication Dispenser Platform (MVP)
Audience: Electronics engineers, firmware developers
JSON convention: Device endpoints under `/api/v1` use **snake_case** property names in JSON (matches backend `JsonPropertyName` attributes). Incoming integration webhooks use **camelCase** (`eventType` , `deviceId`).

---

0. Base URL and authentication

What this section covers: How to address the API and prove the device identity on each request.

Why it matters: Wrong base URL or missing headers produce **401/404** and silent failures in the field; firmware should treat auth as mandatory on every protected call.

Item	Value
API root	`https://<your-host>` (local MVP: `http://localhost:5000`)
Device API base	`{root}/api/v1`
Auth option A	Header `Authorization: Bearer <device_token>` (from registration)
Auth option B	Header `X-API-Key: <key>` (created in Web → Integrations → Device API keys; device must already exist in portal, identified by GUID)

Note: `POST /api/v1/devices/register` has **no** auth. All other device routes below require **A** or **B** unless noted.

Security: Firmware should **always** send `X-API-Key` or `Bearer` on protected routes. Do not rely on the path `{deviceId}` alone.

How auth works (mental model)

- **`Authorization: Bearer …`** — JWT issued by **`POST /devices/register`** (or equivalent auth service). The token carries the device identity the server trusts.
- **`X-API-Key: sk_…`** — Created in the caregiver **Integrations** screen for a **specific** device (GUID). The server maps the key → device; use this when the device was created in the portal first.
- **Path** **`{deviceId}`** — Should be the same cloud GUID as your device record. Always pair it with Bearer or API key so the call is authenticated.

---

1. `GET /api/v1/ping`

Method	`GET`
Path	`/api/v1/ping`
Auth	None

What it does: Returns static health JSON (`status` , `server_time`). **No database writes** — it only proves the HTTP stack and routing work.

Why use it: Cheap way to confirm **network + TLS + correct API hostname** before heartbeats, registration, or OTA logic. Avoids sending large payloads while debugging connectivity.

When to call: After Wi-Fi join, in factory test mode, or before retrying a failed sync.

Response JSON

```
{
  "status": "ok",
  "server_time": "2026-03-26T14:30:00.0000000Z"
}
```

Field	Type	Description
`status`	string	Always `"ok"` when the service is reachable.
`server_time`	string (ISO 8601 UTC)	Server clock; use for rough time sync checks only.

---

2. `POST /api/v1/devices/register`

Method	`POST`
Path	`/api/v1/devices/register`
Auth	None

What it does: Inserts a **new** `Device` row in the database (new internal GUID), stores type/firmware/hardware/MAC, marks the unit online, and returns a **device JWT** (`device_token`) plus suggested `heartbeat_interval` (typically **60** seconds) and `api_endpoint` .

Why use it: Gives firmware a **Bearer token** without a prior portal step — suited to “device calls home first, user links later” flows **if** product uses the **`SMD-` + 8 hex** id scheme.

When *not* to use it: If the caregiver **already created** the device in the web app and you have a **GUID + API key**, skip registration and use **`X-API-Key`** on all `/api/v1/...` calls instead.

Validation: `device_id` must match **`SMD-` + exactly 8 hexadecimal characters** (e.g. `SMD-A1B2C3D4`). Other formats return **`success: false`** (no token).

Request JSON

```
{
  "device_id": "SMD-A1B2C3D4",
  "device_type": "main",
  "firmware_version": "1.0.0",
  "hardware_version": "revB",
  "mac_address": "AA:BB:CC:DD:EE:FF"
}
```

Field	Type	Required	Description
`device_id`	string	Yes	Hardware id: `SMD-` + 8 hex chars.
`device_type`	string	Yes	`main`, `portable`, `smd-100`, `smd-200`, `travel`, etc. Mapped server-side to device type.
`firmware_version`	string	No	Running firmware version string.
`hardware_version`	string	No	PCB / product revision.
`mac_address`	string	No	Wi-Fi MAC for support / inventory.

Response JSON (success)

```
{
  "success": true,
  "device_token": "eyJhbGciOiJIUzI1NiIs...",
  "token_expires_at": "2027-03-26T14:30:00.0000000Z",
  "api_endpoint": "https://your-host/api/v1",
  "heartbeat_interval": 60
}
```

Field	Type	Description
`success`	boolean	`true` when registration succeeded.
`device_token`	string	JWT — send as `Authorization: Bearer ...` on later calls.
`token_expires_at`	string (ISO 8601)	Token expiry (MVP: long-lived, e.g. ~365 days).
`api_endpoint`	string	Base URL for `/api/v1` (server fills host).
`heartbeat_interval`	integer	Suggested seconds between heartbeats (e.g. `60`).

---

3. `POST /api/v1/devices/{deviceId}/heartbeat`

Method	`POST`
Path	`/api/v1/devices/{deviceId}/heartbeat`
Auth	Bearer or `X-API-Key`

What it does: Updates the device row: **last heartbeat**, **online** flags, optional **battery / Wi-Fi / temperature / humidity / firmware** fields. If `slots` is sent, matching **container quantities** are updated by slot number. Very low battery may create **in-app notifications** for the patient.

Why use it: Keeps the cloud (and caregivers) aware the dispenser is **alive**, within **environmental limits**, and shows **inventory** without a separate inventory call. Response includes `next_heartbeat` so firmware can align cadence with server expectations.

When to call: On a **fixed interval** (e.g. every **60** s after registration, or whatever `heartbeat_interval` returned). Also after waking from sleep or reconnecting after outage.

Path `{deviceId}`: Use the cloud **GUID** string when the device exists from the portal (API key flow).

Request JSON

```
{
  "device_id": "550e8400-e29b-41d4-a716-446655440000",
  "timestamp": "2026-03-26T14:30:00.0000000Z",
  "battery": 87,
  "wifi_signal": -52,
  "temperature": 22.5,
  "humidity": 45,
  "firmware": "1.0.0",
  "slots": [
    {
      "slot": 1,
      "medication": "Metformin 500mg",
      "pills": 14
    },
    {
      "slot": 2,
      "medication": "Aspirin 100mg",
      "pills": 30
    }
  ]
}
```

Field	Type	Required	Description
`device_id`	string	Yes	Identifier string for the device (typically same GUID as path).
`timestamp`	string / datetime	No	Device-side time of sample (ISO 8601).
`battery`	integer	No	Percentage **0–100**. Low values may trigger server-side notifications.
`wifi_signal`	integer	No	RSSI in dBm (e.g. `-65`).
`temperature`	number	No	Celsius (or as agreed with product).
`humidity`	integer	No	Relative humidity %.
`firmware`	string	No	Current firmware version reported to cloud.
`slots`	array	No	Per-compartment inventory snapshot.
`slots[].slot`	integer	Yes (in object)	Slot index matching cloud container slot.
`slots[].medication`	string	No	Human-readable name (informational).
`slots[].pills`	integer	Yes (in object)	Count remaining in that slot; updates cloud inventory.

Response JSON

```
{
  "success": true,
  "server_time": "2026-03-26T14:30:01.0000000Z",
  "commands": [],
  "next_heartbeat": 60
}
```

Field	Type	Description
`success`	boolean	`true` if heartbeat applied.
`server_time`	string (ISO 8601)	Server UTC time after processing.
`commands`	array	Reserved for future **remote commands** (MVP often empty).
`next_heartbeat`	integer	Suggested interval in seconds until next heartbeat.

---

4. `GET /api/v1/devices/{deviceId}/schedule`

Method	`GET`
Path	`/api/v1/devices/{deviceId}/schedule`
Auth	Bearer or `X-API-Key`

What it does: Reads **containers + schedules** from the database for this device, filters to schedules **active today** (start/end date), and returns a flat list of **schedule rows** (each row = one time-of-day + weekday bitmask expanded to `mon` … `sun`).

Why use it: Firmware needs authoritative **when to dispense**, **how many pills**, and **which slot** — everything the caregiver configured in the app. Without this, local timers cannot match cloud truth.

When to call: After pairing/linking, after **pull-to-refresh** or a "schedule changed" signal (if you add one later), and periodically (e.g. daily) to pick up edits. Compare count or hash locally to avoid redundant work.

Response JSON

The backend returns **one time string per schedule row** (`TimeOfDay` → `"HH:mm"`). If the patient has the **same medication twice daily**, you will typically see **two `schedules` entries** (same `slot` / `medication`, different `id` / `times`).

`days` are **lowercase three-letter** codes from the server's bitmask (`GetDeviceScheduleQueryHandler`): `mon`, `tue`, `wed`, `thu`, `fri`, `sat`, `sun`.

```
{
  "schedules": [
    {
      "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "medication": "Metformin 500mg",
      "slot": 1,
      "pills": 2,
      "times": ["08:00"],
      "days": ["mon", "wed", "fri"]
    },
    {
      "id": "4fa85f64-5717-4562-b3fc-2c963f66afa7",
      "medication": "Metformin 500mg",
      "slot": 1,
      "pills": 2,
      "times": ["20:00"],
      "days": ["mon", "wed", "fri"]
    }
  ]
}
```

Field	Type	Description
`schedules`	array	Active schedule rows for this device (today within each row's start/end dates).
`schedules[].id`	string (GUID)	That schedule row's id in the database.
`schedules[].medication`	string	From container `MedicationName`.
`schedules[].slot`	integer	Container slot number on the device.
`schedules[].pills`	integer	**Pills per dose** for this row (`container.PillsPerDose`), not total stock.
`schedules[].times`	array of strings	Usually **one** local time `"HH:mm"` per row.
`schedules[].days`	array of strings	**Lowercase** weekday codes: `mon` … `sun`.

---

5. `POST /api/v1/devices/{deviceId}/schedule/confirm`

Method	`POST`
Path	`/api/v1/devices/{deviceId}/schedule/confirm`
Auth	Bearer or `X-API-Key`

What it does: MVP implementation **logs** confirmation and returns `confirmed_at`. Use it as an **audit hook** that the device applied a given schedule snapshot.

Why use it: Support and compliance: proves the dispenser **received** the same number/version of schedule entries the cloud intended — useful when debugging "device didn't dispense" reports.

When to call: Right after a successful `GET .../schedule` parse and NVM write, optionally with a hash or version string in `schedule_version`.

Request JSON

```
{
  "schedule_version": "2026-03-26T1",
  "schedule_count": 4
}
```

Field	Type	Required	Description
`schedule_version`	string	Yes	Your version tag (hash, date, or server-provided id).
`schedule_count`	integer	Yes	Number of schedule items applied locally.

Response JSON

```
{
  "success": true,
  "confirmed_at": "2026-03-26T14:35:00.0000000Z"
}
```

Field	Type	Description
`success`	boolean	Acknowledgement stored.
`confirmed_at`	string (ISO 8601)	Server time of confirmation.

---

6. `POST /api/v1/events`

Method	`POST`
Path	`/api/v1/events`
Auth	Bearer or `X-API-Key`

What it does: Accepts a single **domain event** from the device. The server **logs** it (`DeviceEventLog`), runs the matching handler (update **dispense events**, **container quantities**, **device online/error fields**, **notifications** for patient/caregiver, etc.), then may fire **outgoing webhooks** registered by the user (`device.<event>` style payload). Responds **`202 Accepted`** with an `event_id` .

Why use it: This is the **primary channel** for **semantic** behaviour: “pills dropped”, “patient took dose”, “missed dose”, “refill soon”, motor faults, travel mode, etc. Heartbeat alone does not replace these — events drive adherence logic and integrations.

When to call: At the moment the **physical state** changes (motor finished, door opened, timeout expired), not on a fixed timer — except avoid sending **`HEARTBEAT`** here; use **`POST .../heartbeat`** for periodic telemetry.

Note: Unknown `event` strings yield **400**; auth failure **401**.

Envelope (all events)

```
{
  "event": "DOSE_DISPENSED",
  "device_id": "550e8400-e29b-41d4-a716-446655440000",
  "timestamp": "2026-03-26T08:00:15.0000000Z",
  "data": {}
}
```

Field	Type	Required	Description
`event`	string	Yes	Event type (see table below). **Case-insensitive** on server; uppercase recommended.
`device_id`	string	Yes	Must match authenticated device.
`timestamp`	string / datetime	Yes	When the event occurred (UTC).
`data`	object	No	Payload; shape depends on `event` .

Supported `event` values

`event`	What the server does (summary)
`DOSE_DISPENSED`	Creates **dispense** record, updates **inventory**, may match **schedule** by time; notifies user.

event	What the server does (summary)
`DOSE_TAKEN`	Finds recent **Dispensed** event → **Confirmed**; adherence / notifications.
`DOSE_MISSED`	Marks matching event **Missed**; notifies patient + caregiver.
`REFILL_NEEDED` / `REFILL_CRITICAL`	Updates container qty if present; **low-stock** notifications (critical vs normal).
`DEVICE_ONLINE` / `DEVICE_OFFLINE`	Sets online flags / timestamps; notifications (incl. caregiver on offline).
`DEVICE_ERROR`	Stores last error on device; **error** notification.
`BATTERY_LOW` / `BATTERY_CRITICAL`	Updates battery field; battery notifications.
`TRAVEL_MODE_ON` / `TRAVEL_MODE_OFF`	MVP: mainly **travel started/ended** notifications (payload logged for webhooks).

Note: Prefer `POST .../heartbeat` for periodic telemetry, not `event: HEARTBEAT` on this endpoint.

`data` for `DOSE_DISPENSED`

```
{
  "medication": "Metformin 500mg",
  "slot": 1,
  "pills_dispensed": 2,
  "pills_remaining": 12,
  "scheduled_time": "2026-03-26T08:00:00.0000000Z"
}
```

Field	Type	Required	Description
`medication`	string	Yes	Medication name.
`slot`	integer	Yes	Slot index.
`pills_dispensed`	integer	Yes	Count dispensed this cycle.
`pills_remaining`	integer	Yes	Count left after dispense (updates cloud).
`scheduled_time`	string / datetime	No	Intended dose time (helps match schedule).

`data` for `DOSE_TAKEN`

```
{
  "medication": "Metformin 500mg",
  "pills_taken": 2,
  "seconds_to_take": 120,
  "on_time": true
}
```

Field	Type	Required	Description
`medication`	string	Yes	Medication name.
`pills_taken`	integer	Yes	Count taken.
`seconds_to_take`	integer	No	Seconds from dispense to intake.
`on_time`	boolean	Yes	Whether within expected window.

`data` for `DOSE_MISSED`

```
{
  "medication": "Metformin 500mg",
  "scheduled_time": "2026-03-26T08:00:00.0000000Z",
  "pills_not_taken": 2
}
```

Field	Type	Required	Description
`medication`	string	Yes	Medication name.
`scheduled_time`	string / datetime	Yes	When dose was due.
`pills_not_taken`	integer	No	Missed count.

`data` for `REFILL_NEEDED` / `REFILL_CRITICAL`

```
{
  "medication": "Metformin 500mg",
  "slot": 1,
  "pills_remaining": 4,
  "days_remaining": 2,
  "daily_usage": 2
}
```

Field	Type	Required	Description
`medication`	string	Yes	Medication name.
`slot`	integer	Yes	Slot index.
`pills_remaining`	integer	Yes	Pills left.
`days_remaining`	integer	Yes	Estimated days until empty.
`daily_usage`	integer	No	Pills per day estimate.

`data` for `DEVICE_ONLINE`

```
{
  "firmware": "1.0.0",
  "battery": 90,
  "wifi_signal": -55
}
```

Field	Type	Required	Description
`firmware`	string	Yes	Firmware version string.
`battery`	integer	No	Percentage.
`wifi_signal`	integer	No	RSSI dBm.

`data` for `DEVICE_OFFLINE`

```
{
  "last_seen": "2026-03-26T07:59:00.0000000Z",
  "reason": "user_power_off"
}
```

Field	Type	Required	Description
`last_seen`	string / datetime	Yes	Last known online time.
`reason`	string	No	Short reason code or text.

`data` for `DEVICE_ERROR` (inside `/events`)

```
{
  "error_code": "MOTOR_TIMEOUT",
  "error_type": "hardware",
  "slot": 1,
  "message": "Stepper did not reach home sensor",
  "severity": "high"
}
```

Field	Type	Required	Description
`error_code`	string	Yes	Machine-readable code.
`error_type`	string	Yes	Category (e.g. `hardware`, `sensor`).
`slot`	integer	No	Affected slot if any.
`message`	string	Yes	Human-readable description.
`severity`	string	No	e.g. `low`, `medium`, `high`.

`data` for `BATTERY_LOW` / `BATTERY_CRITICAL`

```
{
  "battery_level": 12,
  "estimated_hours": 4
}
```

Field	Type	Required	Description
`battery_level`	integer	Yes	Percentage 0–100.
`estimated_hours`	integer	No	Estimated runtime remaining.

`data` for `TRAVEL_MODE_ON`

```
{
  "portable_device_id": "660e8400-e29b-41d4-a716-446655440001",
  "medications_transferred": ["Metformin 500mg", "Aspirin 100mg"],
  "days_loaded": 7
}
```

Field	Type	Required	Description
`portable_device_id`	string	Yes	Portable unit id.
`medications_transferred`	array of strings	Yes	Names transferred.
`days_loaded`	integer	Yes	Travel supply duration.

`data` for `TRAVEL_MODE_OFF`

```
{
  "portable_device_id": "660e8400-e29b-41d4-a716-446655440001",
  "days_away": 5,
  "doses_taken": 10,
  "doses_missed": 1
}
```

Field	Type	Required	Description
`portable_device_id`	string	Yes	Portable unit id.
`days_away`	integer	Yes	Days in travel mode.
`doses_taken`	integer	Yes	Doses confirmed.
`doses_missed`	integer	Yes	Doses missed.

Response JSON (`202 Accepted`)

```
{
  "success": true,
  "event_id": "8fa85f64-5717-4562-b3fc-2c963f66afa7"
}
```

Field	Type	Description
`success`	boolean	Event accepted for processing.
`event_id`	string (GUID)	Server log id for this event.

---

7. `POST /api/v1/devices/{deviceId}/inventory`

Method	`POST`
Path	`/api/v1/devices/{deviceId}/inventory`
Auth	Bearer or `X-API-Key`

What it does: For each slot in the body, finds the matching **container** by slot number and sets `Quantity` from `pills`; optionally updates `MedicationName` if `medication` is non-empty. (`capacity` in JSON is **ignored** by the current MVP handler — reserved for future use.)

Why use it: When you do a **full tray recount** (user refill, calibration, or reconciliation) and want to push **all slots at once** without duplicating that data on every heartbeat.

When to call: After **manual refill** flow completes, after **inventory calibration**, or on a **nightly reconciliation** if product requires it.

Request JSON

```
{
  "slots": [
    { "slot": 1, "medication": "Metformin 500mg", "pills": 20, "capacity": 30 },
    { "slot": 2, "medication": "Aspirin 100mg", "pills": 45, "capacity": 60 }
  ]
}
```

Field	Type	Required	Description
`slots`	array	No	Omit or `[]` if nothing to sync.
`slots[].slot`	integer	Yes	Slot index.
`slots[].medication`	string	No	Label.
`slots[].pills`	integer	Yes	Current count.
`slots[].capacity`	integer	No	**Not applied** in current `SyncInventoryCommandHandler`; safe to omit.

Response JSON

```
{
  "success": true,
  "synced_at": "2026-03-26T14:40:00.0000000Z"
}
```

Field	Type	Description
`success`	boolean	Sync applied.
`synced_at`	string (ISO 8601)	Server processing time.

---

8. `POST /api/v1/devices/{deviceId}/error`

Method	`POST`
Path	`/api/v1/devices/{deviceId}/error`
Auth	Bearer or `X-API-Key`

What it does: Records a **device fault** in persistence (event log + device last-error fields), raises **notifications** (patient / caregiver paths per handler), similar outcome to sending `DEVICE_ERROR` on `/events` but via a **dedicated URL** (easier to route in some firmware stacks).

Why use it: Clear separation: **normal events** vs **fault pipeline** (retries, different logging, critical alerts).

When to call: On **non-recoverable** or **user-visible** errors (motor stall, sensor failure, door jam) as soon as detected.

Request JSON

Uses the same shape as `DEVICE_ERROR` `data` (snake_case):

```
{
  "error_code": "SENSOR_FAULT",
  "error_type": "sensor",
  "slot": 2,
  "message": "IR beam blocked",
  "severity": "medium"
}
```

(Field meanings: same as **§6** **`DEVICE_ERROR`**.)

Response JSON

```
{
  "success": true
}
```

---

9. `GET /api/v1/devices/{deviceId}/firmware?current_version=1.0.0`

Method	`GET`
Path	`/api/v1/devices/{deviceId}/firmware`
Query	Optional `current_version` — firmware string you are running
Auth	Bearer or `X-API-Key`

What it does: Returns whether an **OTA package** is available, and metadata (`download_url` , `checksum` , `size_bytes` , `mandatory` , etc.). **MVP controller** currently returns **`update_available: false`** always (placeholder until a firmware registry is wired).

Why use it: Standard **pull** pattern: device asks “is there newer firmware?” before downloading — supports staged rollouts and mandatory updates later.

When to call: After boot, after heartbeat reconnect, or on a **daily** timer — policy is product-specific.

Response JSON (MVP typical)

```
{
  "update_available": false,
  "current_version": "1.0.0",
  "new_version": null,
  "download_url": null,
  "checksum": null,
  "size_bytes": null,
  "release_notes": null,
  "mandatory": false
}
```

Field	Type	Description
`update_available`	boolean	`true` if a build is offered (MVP often `false`).

Field	Type	Description
`current_version`	string	Echo / resolved baseline version.
`new_version`	string / null	Target version if update exists.
`download_url`	string / null	HTTPS URL to firmware image.
`checksum`	string / null	Integrity hash (algorithm per product policy).
`size_bytes`	integer / null	Image size.
`release_notes`	string / null	Text for UI or logs.
`mandatory`	boolean	If `true`, device should prioritize update.

---

10. `POST /api/v1/devices/{deviceId}/firmware/status`

Method	`POST`
Path	`/api/v1/devices/{deviceId}/firmware/status`
Auth	Bearer or `X-API-Key`

What it does: Accepts **progress** updates for an in-flight OTA. When `status` is `completed`, MVP code updates the device's `FirmwareVersion` in the database to `version`. Failures are **logged** with optional `error` text.

Why use it: Support and dashboards need **visibility** (stuck at 40 %, bricked, etc.) without polling the device.

When to call: At state transitions: start download, verify, install, success/fail — throttle high-frequency posts if needed.

Request JSON

```
{
  "status": "downloading",
  "version": "1.1.0",
  "progress": 35,
  "error": null
}
```

Field	Type	Required	Description
`status`	string	Yes	`downloading`, `verifying`, `installing`, `completed`, `failed`.
`version`	string	Yes	Firmware version being installed.
`progress`	integer	No	0–100 % complete.
`error`	string	No	Set when `status` is `failed`.

Response JSON

```
{
  "success": true,
  "received_at": "2026-03-26T14:45:00.0000000Z"
}
```

Field	Type	Description
`success`	boolean	Server accepted the status report.
`received_at`	string (ISO 8601)	UTC time the API recorded the update.

---

11. `POST /api/webhooks/incoming` (integration / minimal firmware)

Method	`POST`
Path	`/api/webhooks/incoming`
Auth	`X-API-Key` **only** (Bearer not used here)
JSON	**camelCase** top-level: `eventType` , `deviceId` , `data`

What it does: Same **business events** as parts of `/api/v1/events` and heartbeat telemetry, but through a **single integration endpoint** used with a **device API key**. Handler updates **device state**, **inventory/dispense** (for dispense types), and **notifications** depending on `eventType` .

Why use it: Minimal JSON parser on MCU, **gateway** that only knows API key, or third-party **bridge** that maps vendor format → our `eventType` + `data` .

When to use vs `/api/v1/events` : Prefer `/api/v1/events` for full device protocol (schedule, heartbeat response fields, richer auth). Use **incoming webhook** when you want **one URL** and **camelCase** envelope.

Caveat: Does **not** replace `GET .../schedule` — still call Device API for schedules if the firmware needs them.

Request JSON

```
{
  "eventType": "heartbeat",
  "deviceId": null,
  "data": {}
}
```

Field	Type	Required	Description
`eventType`	string	Yes	Compared **case-insensitively** then uppercased. Supported in `ProcessIncomingWebhookCommandHandler` includes: `HEARTBEAT` , `DEVICE_ONLINE` , `DEVICE_OFFLINE` , `DEVICE_ERROR` , `DOSE_DISPENSED` , `DOSE_TAKEN` , `DOSE_MISSED` , `REFILL_NEEDED` , `REFILL_CRITICAL` , `BATTERY_LOW` , `BATTERY_CRITICAL` , `TRAVEL_MODE_ON` , `TRAVEL_MODE_OFF` , plus legacy aliases `DEVICE_STATUS` , `DISPENSE_COMPLETED` (same handling as dose dispensed), `LOW_STOCK` .

Field	Type	Required	Description
`deviceId`	string (GUID)	No	Omit to use device bound to API key; if set, must match that device (`WebhooksController` → `403` on mismatch).
`data`	object	No	Shape depends on `eventType`. Telemetry uses **snake_case** keys inside `data` (see below).

`data` shapes for common incoming types

`heartbeat` / `HEARTBEAT` — optional telemetry (read by `UpdateDeviceFromData`):

```
{
  "battery": 87,
  "wifi_signal": -52,
  "temperature": 22.5,
  "humidity": 45,
  "firmware": "1.0.0"
}
```

Field	Type	Description
`battery`	integer	0–100 %.
`wifi_signal`	integer	RSSI (e.g. dBm).
`temperature`	number	Device-reported temperature.
`humidity`	integer	Relative humidity %.
`firmware`	string	Firmware version string.

`dispense_completed` / `DISPENSE_COMPLETED` / `DOSE_DISPENSED` — either **legacy** (GUIDs) or **slot-based** format:

```
{
  "containerId": "770e8400-e29b-41d4-a716-446655440002",
  "scheduleId": "880e8400-e29b-41d4-a716-446655440003"
}
```

or

```
{
  "medication": "Metformin 500mg",
  "slot": 1,
  "pills_dispensed": 2,
  "pills_remaining": 12
}
```

Field	Type	Description
`containerId`	string (GUID)	With `scheduleId`, legacy path: creates dispense event and decrements stock.
`scheduleId`	string (GUID)	Must belong to that container.
`medication`	string	Used in slot-based path.
`slot`	integer	Slot index; server finds container and first matching schedule.

Field	Type	Description
`pills_dispensed`	integer	If `pills_remaining` omitted, stock decreases by this amount.
`pills_remaining`	integer	If set, container quantity set to this value.

`low_stock` / `LOW_STOCK` / **refill types** — `ProcessRefillNeededAsync` reads:

```
{
  "medication": "Metformin 500mg",
  "pills_remaining": 4,
  "days_remaining": 2,
  "slot": 1,
  "containerId": "770e8400-e29b-41d4-a716-446655440002"
}
```

(`containerId` is **camelCase** in JSON; other keys are **snake_case** as in the handler.)

`device_status` / `DEVICE_STATUS` — `data.status` : `"Active"` or `"Paused"` (case-insensitive).

Response

`202 Accepted` on success (`WebhooksController.Incoming`). `401` invalid/missing key; `403` `deviceId` in body does not match key's device; `400` unknown `eventType` or handler returned failure.

---

12. `POST /api/integrations/sync` (bulk / gateway)

Method	`POST`
Path	`/api/integrations/sync`
Auth	`X-API-Key` only
JSON	**camelCase** (`deviceId` , `device` , `events` , …)

What it does: Batch operation: optionally patches **device** fields (`Active` / `Paused` , time zone, last heartbeat). Each valid `events[]` row inserts a `DispenseEvent` and may **decrement container quantity** when status is **Confirmed**. Invalid rows are **skipped** silently.

Why use it: Store-and-forward when the device or gateway is **offline for hours** — replay many doses in one HTTPS session instead of hundreds of `/events` calls.

When to use: LTE/Wi-Fi intermittent, **hub** architecture, or migration tooling — **not** required for real-time dispensers that are always online.

Limitation: `containers` array in the DTO is **not consumed** by current handler — do not rely on it.

Request JSON

```
{
  "deviceId": "550e8400-e29b-41d4-a716-446655440000",
  "device": {
    "status": "Active",
    "timeZoneId": "Europe/Zurich",
    "lastHeartbeatAtUtc": "2026-03-26T14:00:00.0000000Z"
  },
  "events": [
    {
      "id": null,
      "containerId": "770e8400-e29b-41d4-a716-446655440002",
      "scheduleId": "880e8400-e29b-41d4-a716-446655440003",
      "scheduledAtUtc": "2026-03-26T08:00:00.0000000Z",
      "status": "Confirmed",
      "dispensedAtUtc": "2026-03-26T08:01:00.0000000Z",
      "confirmedAtUtc": "2026-03-26T08:05:00.0000000Z"
    }
  ]
}
```

Field	Type	Required	Description
`deviceId`	string	Yes	GUID string; **must equal** `deviceId` resolved from `X-API-Key` (`IntegrationsController` else `400`).
`device`	object	No	If present, updates `Status` , `TimeZoneId` , `LastHeartbeatAtUtc` on the device entity.
`device.status`	string	No	`Active` or `Paused` (case-insensitive per `SyncFromCloudCommandHandler`).
`device.timeZoneId`	string	No	IANA time zone id.
`device.lastHeartbeatAtUtc`	string / datetime	No	Stored on device.
`events`	array	No	Each valid row creates a `DispenseEvent` . Rows with bad GUIDs, wrong device, or missing container/schedule are **skipped** (no error for that row).
`events[].id`	string	No	Present on DTO; **not** used to deduplicate in current handler (server always generates new event id).
`events[].containerId`	string (GUID)	Yes	Must exist and belong to this device.
`events[].scheduleId`	string (GUID)	Yes	Must exist for that container.
`events[].scheduledAtUtc`	string / datetime	Yes	Stored as scheduled time.
`events[].status`	string	Yes	Parsed case-insensitively: `dispensed` , `confirmed` , `missed` , `delayed` , else treated as `pending` .
`events[].dispensedAtUtc`	string / datetime	No	If omitted for non-pending, server may set dispense time to “now”.
`events[].confirmedAtUtc`	string / datetime	No	When marked confirmed.
`containers`	array	No	**Not read** by `SyncFromCloudCommandHandler` in the current codebase — omit; reserved for future use.

Response

`202 Accepted` when `SyncFromCloudCommandHandler` succeeds. `401` invalid key. `400` if body `deviceId` ≠ key's device, or sync handler returns false (e.g. device id not found in DB).

Recommended firmware call order

Step	Call	Reason
1	`GET /api/v1/ping` (optional)	Confirm network path before heavier traffic.
2	`POST /api/v1/devices/register` **or** use portal GUID + API key	Obtain **Bearer** *or* skip if key-only flow.
3	`GET .../schedule`	Load **truth** for alarms and dispense times from cloud.
4	`POST .../schedule/confirm` (optional)	Audit that NVM matches downloaded schedule.
5	`POST .../heartbeat` every `heartbeat_interval`	Liveness + telemetry + slot counts (if not using separate inventory sync).
6	`POST /api/v1/events` (as things happen)	Adherence, dispense, faults — drives notifications and webhooks.
7	`POST .../inventory`	After refill / full recount.
8	`POST .../error`	Structured fault reporting.
9	`GET .../firmware` + `POST .../firmware/status`	OTA check and progress (when product enables real updates).
Alt	`POST /api/webhooks/incoming`	Simpler gateway path with same API key; combine with schedule fetch from Device API as needed.
Alt	`POST /api/integrations/sync`	Bulk catch-up after offline period.

Document verification (source of truth)

This guide was **cross-checked** against the MVP backend in this repository:

Area	Source files
Device routes & auth	`backend/src/Api/Controllers/DeviceApiController.cs`
Request/response DTOs (snake_case)	`backend/src/Application/DTOs/DeviceEventDtos.cs`
Schedule JSON shape	`backend/src/Application/DeviceApi/GetDeviceScheduleQueryHandler.cs`
`/api/v1/events` processing	`backend/src/Application/DeviceApi/ProcessDeviceEventCommandHandler.cs`
Incoming webhook	`backend/src/Api/Controllers/WebhooksController.cs`, `…/Integrations/ProcessIncomingWebhookCommandHandler.cs`

Area	Source files
Bulk sync	`backend/src/Api/Controllers/IntegrationsController.cs`, `…/Integrations/SyncFromCloudCommandHandler.cs`
Registration rules	`backend/src/Application/DeviceApi/RegisterDeviceCommandHandler.cs`

Regenerate HTML/PDF after edits: `python technical-docs/generate_device_api_pdf.py`.

Engineering handoff. For narrative integration steps see root `INTEGRATION.md`.